

Bài 8: NGÔN NGỮ TRUY VẤN DỮ LIỆU (tiếp)

Mục tiêu cần đạt được của bài dạy về kiến thức, kỹ năng

Về kiến thức:

- + Hiểu câu lệnh sắp xếp, kết nối và thống kê dữ liệu
- + Hiểu được cú pháp của lệnh sắp xếp, kết nối và thống kê dữ liệu trong các trường hợp khác nhau.

Về kỹ năng:

- + Thực hiện Sắp xếp kết quả truy vấn
- + Thực hiện phép kết nối các bảng trong truy vấn dữ liệu
- + Thực hiện thống kê dữ liệu trong câu lệnh truy vấn.

8.1. Sắp xếp kết quả truy vấn với ORDER BY trong SQL

Mặc định các dòng dữ liệu trong kết quả của câu truy vấn tuân theo thứ tự của chúng trong bảng dữ liệu hoặc được sắp xếp theo chỉ mục (nếu trên bảng có chỉ mục). Trong trường hợp muốn dữ liệu được sắp xếp theo chiều tăng hoặc giảm của giá trị của một hoặc nhiều trường, ta sử dụng thêm mệnh đề ORDER BY trong câu lệnh SELECT.

8.1.1. Giới thiệu về ORDER BY

Trong SQL, ORDER BY được sử dụng để sắp xếp kết quả của một truy vấn theo thứ tự tăng dần (ASC) hoặc giảm dần (DESC) dựa trên một hoặc nhiều cột. Mặc định, kết quả sẽ được sắp xếp theo thứ tự tăng dần nếu không chỉ định cụ thể.

Sau ORDER BY là danh sách các cột cần sắp xếp (tối đa là 16 cột).

Sắp xếp giúp dữ liệu trở nên trực quan hơn và dễ tìm kiếm thông tin cần thiết trong các bảng dữ liệu lớn.

8.1.2. Cú pháp của ORDER BY

SELECT [Danh sách các cột]

FROM [Tên bảng]

WHERE [Điều kiện lọc (nếu có)]

ORDER BY [Tên cột1] [**ASC|DESC**], [Tên cột2] [**ASC|DESC**], ...;

Trong đó:

- ASC: Thứ tự tăng dần (mặc định).
- DESC: Thứ tự giảm dần.

Ví dụ 1: Sắp xếp theo một cột

Yêu cầu: Lấy danh sách các hàng hóa và sắp xếp theo tên hàng (TenH) theo thứ tự tăng dần.

Câu lệnh SQL:

```
SELECT MaH, TenH, DVT, SoLuong
FROM Hang
ORDER BY TenH ASC;
```

Giải thích:

- **ORDER BY TenH ASC:** Sắp xếp kết quả theo tên hàng (TenH) từ A đến Z.

Ví dụ 2: Sắp xếp theo nhiều cột

Yêu cầu: Lấy danh sách các hóa đơn và sắp xếp theo ngày lập hóa đơn (Ngày) từ mới nhất đến cũ nhất. Nếu ngày lập hóa đơn giống nhau, sắp xếp theo số hóa đơn (SoHD) theo thứ tự tăng dần.

Câu lệnh SQL:

```
SELECT SoHD, Ngày, MaK
FROM HoaDon
ORDER BY Ngày DESC, SoHD ASC;
```

Giải thích:

- **ORDER BY Ngày DESC, SoHD ASC:** Đầu tiên sắp xếp theo ngày lập hóa đơn từ mới nhất đến cũ nhất, sau đó nếu ngày giống nhau, sắp xếp theo số hóa đơn từ nhỏ đến lớn.

Ví dụ 3: Sắp xếp theo thứ tự giảm dần

Yêu cầu: Lấy danh sách các hàng hóa và số lượng còn lại (SoLuong), sắp xếp theo số lượng còn lại giảm dần.

Câu lệnh SQL:

```
SELECT MaH, TenH, DVT, SoLuong
```

FROM Hang

ORDER BY SoLuong DESC;

Giải thích:

- ORDER BY SoLuong DESC: Sắp xếp kết quả theo số lượng hàng hóa từ nhiều đến ít.

Ví dụ 4: Câu lệnh dưới đây hiển thị danh sách các hoá đơn và sắp xếp theo chiều giảm dần của số lượng sản phẩm

SELECT * FROM ChiTietHoaDon

ORDER BY SoLuong **DESC**

Ví dụ 5: Sắp xếp theo dữ liệu ngày tháng

Yêu cầu: Lấy danh sách các hóa đơn và tên khách hàng, sắp xếp theo ngày lập hóa đơn (Ngày) từ gần đây nhất đến xa nhất.

Câu lệnh SQL:

SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.TenK

FROM HoaDon

INNER JOIN Khach **ON** HoaDon.MaK = Khach.MaK

ORDER BY HoaDon.Ngay DESC;

Giải thích:

- ORDER BY HoaDon.Ngay DESC: Sắp xếp kết quả theo ngày lập hóa đơn từ gần đây nhất đến xa nhất.

Chú ý: Nếu sau ORDER BY có nhiều cột thì việc sắp xếp dữ liệu sẽ được ưu tiên theo chiều từ trái qua phải

Ví dụ 6: Liệt kê danh sách khách hàng và sắp xếp tên khách theo Alphabet, nếu trùng tên thì sắp theo địa chỉ

SELECT * FROM Khach

ORDER BY TenK, DiaChi

8.1.3. Kết hợp ORDER BY với WHERE

Yêu cầu: Lấy danh sách các chi tiết hóa đơn với số lượng lớn hơn 10 và sắp xếp theo đơn giá (DonGia) từ cao đến thấp.

Câu lệnh SQL:

```
SELECT SoHD, MaH, SoLuong, DonGia
FROM ChiTietHoaDon
WHERE SoLuong > 10
ORDER BY DonGia DESC;
```

Giải thích:

- WHERE SoLuong > 10: Lọc những chi tiết hóa đơn có số lượng lớn hơn 10.
- ORDER BY DonGia DESC: Sắp xếp kết quả theo đơn giá từ cao đến thấp.

8.1.4. Sắp xếp theo biểu thức tính toán

Yêu cầu: Lấy danh sách các hàng hóa và tính tổng giá trị tồn kho (SoLuong * DonGia) và sắp xếp theo tổng giá trị tồn kho từ cao đến thấp.

Câu lệnh SQL:

```
SELECT Hang.MaH, Hang.TenH, Hang.SoLuong, ChiTietHoaDon.DonGia,
(Hang.SoLuong * ChiTietHoaDon.DonGia) AS TotalValue
FROM Hang
INNER JOIN ChiTietHoaDon ON Hang.MaH = ChiTietHoaDon.MaH
ORDER BY TotalValue DESC;
```

Giải thích:

- (Hang.SoLuong * ChiTietHoaDon.DonGia): Tính toán tổng giá trị tồn kho của hàng hóa.
- ORDER BY TotalValue DESC: Sắp xếp kết quả theo tổng giá trị tồn kho từ cao đến thấp.

8.1.5. Sắp xếp với các hàm tổng hợp

Yêu cầu: Lấy danh sách các hàng hóa và số lượng bán ra (sử dụng tổng số lượng từ bảng ChiTietHoaDon), sắp xếp theo số lượng bán ra giảm dần.

Câu lệnh SQL:

```
SELECT Hang.MaH, Hang.TenH, SUM(ChiTietHoaDon.SoLuong) AS  
TotalSold  
  
FROM Hang  
  
INNER JOIN ChiTietHoaDon ON Hang.MaH = ChiTietHoaDon.MaH  
  
GROUP BY Hang.MaH, Hang.TenH  
  
ORDER BY TotalSold DESC;
```

Giải thích:

- SUM(ChiTietHoaDon.SoLuong): Tính tổng số lượng hàng hóa bán ra.
- ORDER BY TotalSold DESC: Sắp xếp kết quả theo số lượng bán ra từ cao đến thấp.

8.1.6. Nhận xét

- ORDER BY là một công cụ mạnh mẽ trong SQL giúp chúng ta sắp xếp dữ liệu theo yêu cầu.
- Sử dụng ORDER BY với các cột cụ thể hoặc biểu thức tính toán giúp dễ dàng phân tích và báo cáo dữ liệu.
- Kết hợp với WHERE để lọc dữ liệu trước khi sắp xếp, và sử dụng các hàm tổng hợp để phân tích dữ liệu chi tiết hơn.

Bài tập Thực hành: Bài toán quản lý bán hàng

- Lấy danh sách các khách hàng và sắp xếp theo tên khách hàng (TenK) theo thứ tự giảm dần.
- Lấy danh sách các chi tiết hóa đơn với số lượng nhỏ hơn 5 và sắp xếp theo số lượng tăng dần.
- Lấy danh sách các hóa đơn và tổng giá trị hóa đơn (tính tổng từ bảng ChiTietHoaDon), sắp xếp theo tổng giá trị hóa đơn từ cao đến thấp.

8.2. Phép kết nối

Trong cơ sở dữ liệu quan hệ, việc lấy dữ liệu từ nhiều bảng có liên quan là một thao tác phổ biến. Để thực hiện điều này, chúng ta sử dụng phép kết nối (JOIN) giữa các bảng.

Có hai cách chính để thực hiện kết nối:

- Sử dụng điều kiện kết nối bằng trong mệnh đề WHERE.
- Sử dụng cú pháp JOIN trong mệnh đề FROM.

8.2.1. Sử dụng điều kiện kết nối bằng trong mệnh đề WHERE

Trong các hệ quản trị cơ sở dữ liệu quan hệ như SQL Server, phép kết nối là một công cụ quan trọng để lấy dữ liệu từ nhiều bảng có liên quan. Một cách để thực hiện phép kết nối giữa các bảng là sử dụng điều kiện kết nối bằng trong mệnh đề WHERE. Đây là một phương pháp truyền thống trước khi cú pháp hiện đại của JOIN được giới thiệu (như INNER JOIN, LEFT JOIN, ...).

Khi cần thực hiện một yêu cầu truy vấn dữ liệu từ hai hay nhiều bảng, ta phải sử dụng đến phép kết nối. Để thực hiện được một phép nối, cần phải xác định được những yếu tố sau:

- Những cột nào cần hiển thị trong kết quả truy vấn.
- Những bảng nào có tham gia vào truy vấn.
- Điều kiện để thực hiện phép nối giữa các bảng dữ liệu là gì?

Trong các yếu tố kể trên, việc xác định chính xác điều kiện để thực hiện phép nối giữa các bảng đóng vai trò quan trọng nhất. Trong đa số các trường hợp, điều kiện của phép nối được xác định nhờ vào mối quan hệ giữa các bảng cần phải truy xuất dữ liệu.

Thông thường, đó là điều kiện bằng nhau giữa khoá chính và khóa ngoài của hai bảng có quan hệ với nhau.

1. Khái niệm về Kết nối bằng (Equi-Join)

Kết nối bằng là phép kết nối giữa hai bảng dựa trên một điều kiện, thông thường là điều kiện so sánh bằng giữa các cột của hai bảng có liên quan. Điều kiện này có thể được viết trong mệnh đề WHERE của câu lệnh SELECT để thực hiện phép nối giữa hai bảng.

Ví dụ: Giả sử có hai bảng Khách và HoaDon. Để lấy thông tin về khách hàng và các đơn hàng của họ, chúng ta cần kết nối hai bảng này dựa trên cột MaK (có trong cả hai bảng).

2. Cú pháp sử dụng Điều kiện Kết nối bằng trong mệnh đề WHERE

Cú pháp để thực hiện phép kết nối bằng trong mệnh đề WHERE như sau:

```
SELECT column_name(s)

FROM table1, table2

WHERE table1.column_name = table2.column_name;
```

Trong đó:

- table1 và table2: Là hai bảng mà chúng ta muốn kết nối.
- column_name: Là cột khóa có liên quan trong cả hai bảng mà phép kết nối sẽ dựa trên nó.

Chú ý:

- Tên của một số cột nào đó trong các bảng có tham gia vào truy vấn. Nếu tên cột trong các bảng trùng tên nhau thì tên cột phải được viết dưới dạng: **Tên_bảng.tên_cột**
- Dấu sao (*) được sử dụng trong danh sách chọn khi cần hiển thị tất cả các cột của các bảng tham gia truy vấn.
- Trong trường hợp cần hiển thị tất cả các cột của một bảng nào đó, ta sử dụng cách viết: **tên_bảng.***

3. Ví dụ

Ví dụ: Lấy danh sách các hóa đơn cùng với tên khách hàng đã mua hàng.

Thực hiện: Kết nối bảng **HoaDon** và bảng **Khach** dựa trên cột **MaK**.

Câu lệnh SQL:

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.TenK

FROM HoaDon, Khach

WHERE HoaDon.MaK = Khach.MaK;
```

Giải thích:

- **FROM** HoaDon, Khach: Chỉ định lấy dữ liệu từ hai bảng HoaDon và Khach.
- **WHERE** HoaDon.MaK = Khach.MaK: Điều kiện kết nối bằng giữa hai bảng.

Kết quả: Danh sách các hóa đơn kèm theo tên khách hàng.

Ví dụ: Liệt kê danh sách khách hàng đã mua hàng có địa chỉ tại ‘Thái Nguyên’

```
SELECT Khach.MaK, Khach.TenK, Khach.DiaChi, Khach.SDT  
FROM Khach, HoaDon  
WHERE Khach.MaK = HoaDon.MaK  
AND Khach.DiaChi = N‘Thái Nguyên’
```

hoặc viết dưới dạng ngắn gọn hơn:

```
SELECT Khach.*  
FROM Khach, HoaDon  
WHERE Khach.MaK = HoaDon.MaK  
AND Khach.DiaChi = N‘Thái Nguyên’
```

4. Kết nối Nhiều Bảng

Yêu cầu: Lấy thông tin chi tiết về các hóa đơn, bao gồm số hóa đơn, ngày, tên khách hàng, tên hàng, số lượng và đơn giá.

Thực hiện: Kết nối các bảng HoaDon, Khach, ChiTietHoaDon, và Hang.

Câu lệnh SQL:

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.TenK, Hang.TenH,  
ChiTietHoaDon.SoLuong, ChiTietHoaDon.DonGia  
FROM HoaDon, Khach, ChiTietHoaDon, Hang  
WHERE HoaDon.MaK = Khach.MaK  
AND HoaDon.SoHD = ChiTietHoaDon.SoHD  
AND ChiTietHoaDon.MaH = Hang.MaH;
```

Giải thích:

- **WHERE:** Chứa các điều kiện kết nối giữa các bảng.
- HoaDon.MaK = Khach.MaK: Kết nối HoaDon với Khach.
- HoaDon.SoHD = ChiTietHoaDon.SoHD: Kết nối HoaDon với ChiTietHoaDon.
- ChiTietHoaDon.MaH = Hang.MaH: Kết nối ChiTietHoaDon với Hang.

5. Ưu điểm và Nhược điểm của Điều kiện Kết nối bằng trong WHERE

Ưu điểm:

- **Đơn giản:** Cú pháp dễ hiểu và dễ viết, đặc biệt với những người mới học SQL.
- **Tương thích:** Là phương pháp truyền thống, có thể sử dụng trong hầu hết các hệ quản trị CSDL và các phiên bản cũ của SQL Server.

Nhược điểm:

- **Thiếu tính rõ ràng:** Việc sử dụng điều kiện kết nối trong mệnh đề WHERE có thể làm cho câu truy vấn dài và khó đọc khi có nhiều bảng và nhiều điều kiện.
- **Khó mở rộng:** Khi cần sử dụng các phép kết nối phức tạp hơn như LEFT JOIN, RIGHT JOIN, thì phương pháp này trở nên kém hiệu quả và khó duy trì.
- **Dễ gây nhầm lẫn:** Khi sử dụng điều kiện kết nối cùng với các điều kiện lọc khác trong mệnh đề WHERE, rất dễ nhầm lẫn giữa điều kiện kết nối và điều kiện lọc dữ liệu.

8.2.2. Sử dụng phép kết nối (JOIN) trong SQL server

1. Tổng quan về phép kết nối (JOIN)

Phép kết nối (JOIN) là cách kết hợp dữ liệu từ nhiều bảng dựa trên một điều kiện cụ thể. Trong hệ quản trị CSDL, các bảng thường có mối quan hệ với nhau qua các khóa (key), và phép kết nối cho phép truy xuất dữ liệu từ nhiều bảng liên quan.

Cú pháp JOIN cho phép kết nối các bảng một cách rõ ràng và linh hoạt hơn.

Các loại JOIN phổ biến:

INNER JOIN

LEFT JOIN

RIGHT JOIN

FULL OUTER JOIN

Cú pháp tổng quát:

SELECT [Danh sách cột]

FROM Bảng1

[LOẠI JOIN] Bảng2 **ON** Bảng1.Cột_Kết_Nối = Bảng2.Cột_Kết_Nối;

Ví dụ: Lấy danh sách các hóa đơn cùng với tên khách hàng đã mua hàng.

Sử dụng INNER JOIN:

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.TenK
```

```
FROM HoaDon
```

```
INNER JOIN Khach ON HoaDon.MaK = Khach.MaK;
```

Giải thích:

- FROM HoaDon INNER JOIN Khach: Thực hiện INNER JOIN giữa HoaDon và Khach.
- ON HoaDon.MaK = Khach.MaK: Điều kiện kết nối.

2. Các loại phép kết nối trong SQL Server

INNER JOIN

Định nghĩa: Trả về các bản ghi khi có sự khớp dữ liệu trong cả hai bảng. Nếu không có bản ghi nào khớp, thì nó sẽ không trả về gì.

Cú pháp:

```
SELECT column_name(s)
```

```
FROM table1
```

```
INNER JOIN table2 ON table1.column_name = table2.column_name;
```

Ví dụ: Lấy thông tin chi tiết về các hóa đơn, bao gồm số hóa đơn, ngày, tên khách hàng, tên hàng, số lượng và đơn giá.

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.TenK, Hang.TenH,  
ChiTietHoaDon.SoLuong, ChiTietHoaDon.DonGia
```

```
FROM HoaDon
```

```
INNER JOIN Khach ON HoaDon.MaK = Khach.MaK
```

```
INNER JOIN ChiTietHoaDon ON HoaDon.SoHD = ChiTietHoaDon.SoHD
```

```
INNER JOIN Hang ON ChiTietHoaDon.MaH = Hang.MaH;
```

LEFT JOIN (LEFT OUTER JOIN)

Định nghĩa: Trả về tất cả các bản ghi từ bảng bên trái và các bản ghi khớp từ bảng bên phải. Nếu không có giá trị khớp từ bảng bên phải, kết quả sẽ là NULL cho cột của bảng bên phải.

Cú pháp:

```
SELECT column_name(s)
```

```
FROM table1
```

```
LEFT JOIN table2 ON table1.column_name = table2.column_name;
```

Ví dụ: Lấy danh sách tất cả các khách hàng và hóa đơn của họ. Nếu khách hàng chưa có hóa đơn, hiển thị NULL cho thông tin hóa đơn.

Câu lệnh SQL:

```
SELECT Khach.MaK, Khach.TenK, HoaDon.SoHD, HoaDon.Ngay
```

```
FROM Khach
```

```
LEFT JOIN HoaDon ON Khach.MaK = HoaDon.MaK;
```

Giải thích:

- **LEFT JOIN:** Lấy tất cả các bản ghi từ bảng bên trái (Khach) và các bản ghi khớp từ bảng bên phải (HoaDon).

Ví dụ: Danh sách tất cả sinh viên, kèm theo thông tin khóa học (nếu có). Những sinh viên chưa đăng ký khóa học sẽ có cột khóa học là NULL.

```
SELECT students.name, courses.course_name
```

```
FROM students
```

```
LEFT JOIN courses ON students.course_id = courses.id;
```

RIGHT JOIN (RIGHT OUTER JOIN)

Định nghĩa: Trả về tất cả các bản ghi từ bảng bên phải và các bản ghi khớp từ bảng bên trái. Nếu không có giá trị khớp từ bảng bên trái, kết quả sẽ là NULL cho cột của bảng bên trái.

Cú pháp:

```
SELECT column_name(s)
```

```
FROM table1
```

```
RIGHT JOIN table2 ON table1.column_name = table2.column_name;
```

Ví dụ: Lấy danh sách tất cả các hóa đơn và thông tin khách hàng. Nếu hóa đơn không có khách hàng tương ứng, hiển thị NULL cho thông tin khách hàng.

Câu lệnh SQL:

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.MaK, Khach.TenK  
  
FROM HoaDon  
  
RIGHT JOIN Khach ON HoaDon.MaK = Khach.MaK;
```

Lưu ý: Trong thực tế, RIGHT JOIN ít được sử dụng hơn LEFT JOIN. Câu truy vấn trên có thể được viết lại bằng LEFT JOIN bằng cách đổi thứ tự bảng.

Ví dụ: Danh sách tất cả các khóa học, kèm theo sinh viên tham gia khóa học (nếu có). Nếu không có sinh viên đăng ký, cột tên sinh viên sẽ là NULL.

```
SELECT students.name, courses.course_name  
  
FROM students  
  
RIGHT JOIN courses ON students.course_id = courses.id;
```

FULL JOIN (FULL OUTER JOIN)

Định nghĩa: Trả về tất cả các bản ghi khi có sự khớp ở một trong hai bảng. Nếu không có khớp trong một bảng, giá trị NULL sẽ được trả về cho cột tương ứng.

Cú pháp:

```
SELECT column_name(s)  
  
FROM table1  
  
FULL JOIN table2 ON table1.column_name = table2.column_name;
```

Ví dụ: Lấy danh sách tất cả các khách hàng và hóa đơn, bao gồm cả những khách hàng chưa có hóa đơn và những hóa đơn không có khách hàng.

Câu lệnh SQL:

```
SELECT Khach.MaK, Khach.TenK, HoaDon.SoHD, HoaDon.Ngay  
  
FROM Khach  
  
FULL OUTER JOIN HoaDon ON Khach.MaK = HoaDon.MaK;
```

Ví dụ: Danh sách tất cả sinh viên và khóa học, bao gồm cả những sinh viên chưa đăng ký khóa học và những khóa học chưa có sinh viên tham gia.

```
SELECT students.name, courses.course_name  
  
FROM students  
  
FULL OUTER JOIN courses ON students.course_id = courses.id;
```

3. Kết hợp Điều kiện Lọc Dữ Liệu

Yêu cầu: Lấy danh sách các hóa đơn có ngày lập sau ngày 01/01/2023 cùng với tên khách hàng.

Câu lệnh SQL Sử dụng INNER JOIN và WHERE:

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.TenK
FROM HoaDon INNER JOIN Khach ON HoaDon.MaK = Khach.MaK
WHERE HoaDon.Ngay > '2023-01-01';
```

Giải thích:

- INNER JOIN: Kết nối HoaDon và Khach.
- WHERE HoaDon.Ngay > '2023-01-01': Điều kiện lọc dữ liệu.

4. So sánh giữa Kết nối bằng trong WHERE và JOIN

Điểm Chung

- Cả hai phương pháp đều cho phép kết nối và lấy dữ liệu từ nhiều bảng.
- Kết quả truy vấn có thể giống nhau khi sử dụng INNER JOIN và điều kiện kết nối trong WHERE.

Điểm khác biệt

- Trong các phiên bản hiện đại của SQL, cú pháp JOIN đã thay thế phần lớn việc sử dụng điều kiện kết nối bằng trong mệnh đề WHERE do những ưu điểm của nó:
- Cú pháp JOIN rõ ràng hơn: Với cú pháp JOIN, chúng ta có thể dễ dàng phân biệt được điều kiện kết nối (nằm trong mệnh đề ON) và các điều kiện lọc dữ liệu (nằm trong mệnh đề WHERE).
- Hỗ trợ các loại JOIN phức tạp: Cú pháp JOIN cho phép chúng ta thực hiện các loại kết nối phức tạp hơn như LEFT JOIN, RIGHT JOIN, và FULL JOIN, điều mà điều kiện kết nối bằng trong mệnh đề WHERE không hỗ trợ rõ ràng.

5. Ví dụ So sánh: INNER JOIN và Điều kiện Kết nối bằng trong WHERE

Cách 1: Sử dụng INNER JOIN

```
SELECT Customers.customer_name, Orders.order_id, Orders.order_date
FROM Customers
```

INNER JOIN Orders **ON** Customers.customer_id = Orders.customer_id;

Cách 2: Sử dụng Điều kiện Kết nối bằng trong WHERE

SELECT Customers.customer_name, Orders.order_id, Orders.order_date

FROM Customers, Orders

WHERE Customers.customer_id = Orders.customer_id;

Kết quả của hai câu truy vấn này là giống nhau, nhưng cú pháp INNER JOIN rõ ràng và dễ bảo trì hơn.

6. Lưu ý khi Sử dụng Điều kiện Kết nối bằng trong WHERE

- Sự rõ ràng trong câu truy vấn: Nếu truy vấn của chúng ta chỉ đơn giản là lấy dữ liệu từ hai bảng, phương pháp sử dụng điều kiện kết nối bằng trong WHERE có thể được chấp nhận. Tuy nhiên, khi chúng ta cần thực hiện các truy vấn phức tạp hơn, hãy cân nhắc sử dụng cú pháp JOIN để dễ hiểu hơn.
- Tránh sự nhầm lẫn với điều kiện lọc: Mệnh đề WHERE thường được sử dụng để lọc dữ liệu, do đó, hãy cẩn thận để không nhầm lẫn điều kiện kết nối với các điều kiện lọc dữ liệu khác.

8.2.3. ví dụ

1. Ghi nhận sinh viên đăng ký một khóa học

Giả sử có một sinh viên với StudentID = 1 muốn đăng ký vào khóa học có CourseID = 2. Chúng ta có thể thực hiện câu truy vấn sau để ghi nhận thông tin này vào bảng **Enrollments**.

INSERT INTO Enrollments (StudentID, CourseID, EnrollmentDate)

VALUES (1, 2, '2024-09-07');

2. Lấy thông tin về tất cả sinh viên đăng ký vào một khóa học

SELECT Students.StudentID, Students.FirstName, Students.LastName,
Enrollments.EnrollmentDate, Enrollments.Grade

FROM Enrollments

INNER JOIN Students **ON** Enrollments.StudentID = Students.StudentID

WHERE Enrollments.CourseID = 2;

3. Lấy danh sách các khóa học mà một sinh viên đã đăng ký

```

SELECT          Courses.CourseID,          Courses.CourseName,
Enrollments.EnrollmentDate, Enrollments.Grade

FROM Enrollments

INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID

WHERE Enrollments.StudentID = 1;

```

Sử dụng INNER JOIN

4. Viết một truy vấn để lấy danh sách tất cả các sinh viên và tên khóa học mà họ đã đăng ký.

```

SELECT students.name, courses.course_name

FROM students

INNER JOIN courses ON students.course_id = courses.id;

```

Sử dụng LEFT JOIN

5. Viết truy vấn để lấy danh sách tất cả sinh viên và khóa học mà họ đăng ký. Nếu sinh viên chưa đăng ký khóa học, hãy hiển thị NULL ở cột khóa học.

```

SELECT students.name, courses.course_name

FROM students

LEFT JOIN courses ON students.course_id = courses.id;

```

Sử dụng FULL OUTER JOIN

6. Viết truy vấn để lấy danh sách tất cả sinh viên và khóa học, bao gồm cả những sinh viên chưa đăng ký khóa học và những khóa học chưa có sinh viên tham gia.

```

SELECT students.name, courses.course_name

FROM students

FULL OUTER JOIN courses ON students.course_id = courses.id;

```

8.3. Thống kê dữ liệu với GROUP BY

8.3.1. Giới thiệu về GROUP BY

Trong SQL, Mệnh đề GROUP BY trong câu lệnh SELECT được sử dụng để nhóm các bản ghi có cùng giá trị trong một hoặc nhiều cột lại với nhau và thực hiện các phép toán tổng hợp trên các nhóm đó. Khi sử dụng GROUP BY, chúng ta thường kết

hợp với các hàm tổng hợp như SUM(), COUNT(), AVG(), MAX(), MIN() để tính toán dữ liệu trong mỗi nhóm.

Các hàm nhóm được sử dụng để tính giá trị thống kê cho toàn bảng hoặc trên mỗi nhóm dữ liệu. Chúng có thể được sử dụng như là các cột trong danh sách chọn của câu lệnh SELECT hoặc xuất hiện trong mệnh đề HAVING, nhưng không được phép xuất hiện trong mệnh đề WHERE

SQL cung cấp các hàm nhóm dưới đây:

Hàm nhóm	Chức năng
SUM(biểu_thức)	Tính tổng các giá trị
AVG(biểu_thức)	Tính trung bình của các giá trị
COUNT(biểu_thức)	Đếm số các giá trị trong biểu thức
COUNT(*)	Đếm số các dòng được chọn
MAX(biểu_thức)	Tính giá trị lớn nhất
MIN(biểu_thức)	Tính giá trị nhỏ nhất

Trong đó:

- Hàm SUM, AVG chỉ làm việc với các biểu thức số
- Hàm SUM, AVG, COUNT, MIN và MAX bỏ qua các giá trị NULL khi tính toán.
- Hàm COUNT(*) không bỏ qua các giá trị NULL

8.3.2. Cú pháp của GROUP BY

SELECT [Danh sách các cột], [Hàm tổng hợp]

FROM [Tên bảng]

WHERE [Điều kiện lọc (nếu có)]

GROUP BY [Tên cột1], [Tên cột2], ...;

Trong đó:

- Danh sách các cột: Cột hoặc các cột chúng ta muốn hiển thị trong kết quả.

- Hàm tổng hợp: Các hàm như SUM(), COUNT(), AVG(), MAX(), MIN(), v.v.
- GROUP BY: Cột hoặc các cột dùng để nhóm dữ liệu.

8.3.3. Thống kê trên toàn bộ dữ liệu

Khi cần tính toán giá trị thống kê trên toàn bộ dữ liệu, ta sử dụng các hàm nhóm trong danh sách chọn của câu lệnh SELECT. Trong trường hợp này, trong danh sách chọn không được sử dụng bất kỳ một tên cột hay biểu thức nào ngoài các hàm gộp.

Ví dụ: Để thống kê giá trung bình của tất cả các hoá đơn trong bảng chi tiết hoá đơn ta sử dụng câu lệnh như sau:

```
SELECT AVG(DonGia)  
  
FROM ChiTietHoaDon
```

8.3.4. Thống kê dữ liệu trên các nhóm

Trong trường hợp cần thực hiện tính toán các giá trị thống kê trên các nhóm dữ liệu, ta sử dụng mệnh đề GROUP BY để phân hoạch dữ liệu vào trong các nhóm. Các hàm nhóm được sử dụng sẽ thực hiện thao tác tính toán trên mỗi nhóm và cho biết giá trị thống kê theo các nhóm dữ liệu.

Ví dụ: Câu lệnh dưới đây cho biết số lần mua hàng của mỗi khách hàng

```
SELECT Khach.MaK, Khach.TenK, COUNT(Khach.MaK) AS N'Số lần mua hàng'  
  
FROM Khach, HoaDon  
  
WHERE Khach.MaK = HoaDon.MaK  
  
GROUP BY Khach.MaK, Khach.TenK
```

Chú ý:

- Biểu thức nào điều khiển việc phân nhóm dữ liệu thì các biểu thức đó phải được liệt kê sau mệnh đề GROUP BY.
- Trong trường hợp danh sách chọn của câu lệnh SELECT có các hàm nhóm và những biểu thức không phải là đối số của các hàm nhóm thì những biểu thức này phải được liệt kê đầy đủ trong mệnh đề GROUP BY, nếu không câu lệnh sẽ không hợp lệ

Ví dụ: Dưới đây là một câu lệnh sai do thiếu trường TenK sau mệnh đề GROUP BY

```

SELECT Khach.MaK, Khach.TenK, COUNT(Khach.MaK) AS N'Số lần mua
hàng'

FROM Khach, HoaDon

WHERE Khach.MaK = HoaDon.MaK

GROUP BY Khach.MaK

```

8.3.5. Ví dụ về GROUP BY

Sử dụng cơ sở dữ liệu quản lý bán hàng, gồm các bảng Hang, Khach, HoaDon, và ChiTietHoaDon.

Ví dụ 1: Tính tổng số lượng của mỗi loại hàng hóa theo mã hàng.

Câu lệnh SQL:

```

SELECT MaH, SUM(SoLuong) AS TotalQuantity

FROM Hang

GROUP BY MaH;

```

Giải thích:

- SUM(SoLuong): Tính tổng số lượng của mỗi loại hàng hóa.
- GROUP BY MaH: Nhóm dữ liệu theo mã hàng (MaH).

Ví dụ 2: Tính tổng doanh thu của mỗi hóa đơn bằng cách nhân số lượng và đơn giá, và nhóm theo số hóa đơn.

Câu lệnh SQL:

```

SELECT SoHD, SUM(SoLuong * DonGia) AS TotalRevenue

FROM ChiTietHoaDon

GROUP BY SoHD;

```

Giải thích:

- SUM(SoLuong * DonGia): Tính tổng doanh thu của mỗi hóa đơn.
- GROUP BY SoHD: Nhóm dữ liệu theo số hóa đơn (SoHD).

Ví dụ 3: Đếm số lượng hóa đơn của mỗi khách hàng.

Câu lệnh SQL:

```

SELECT MaK, COUNT(SoHD) AS NumberOfInvoices
FROM HoaDon
GROUP BY MaK;

```

Giải thích:

- COUNT(SoHD): Đếm số lượng hóa đơn của mỗi khách hàng.
- GROUP BY MaK: Nhóm dữ liệu theo mã khách hàng (MaK).

Ví dụ 4: Tính giá trị đơn hàng cao nhất và thấp nhất của mỗi loại hàng hóa.

Câu lệnh SQL:

```

SELECT MaH, MAX(DonGia) AS MaxPrice, MIN(DonGia) AS MinPrice
FROM ChiTietHoaDon
GROUP BY MaH;

```

Giải thích:

- MAX(DonGia): Tính giá đơn hàng cao nhất.
- MIN(DonGia): Tính giá đơn hàng thấp nhất.
- GROUP BY MaH: Nhóm dữ liệu theo mã hàng (MaH).

Ví dụ 5. Kết hợp GROUP BY với JOIN

Yêu cầu: Tính tổng doanh thu của mỗi khách hàng và nhóm theo mã khách hàng.

Câu lệnh SQL:

```

SELECT  Khách.MaK,  Khách.TenK,  SUM(ChiTietHoaDon.SoLuong *
ChiTietHoaDon.DonGia) AS TotalRevenue
FROM ChiTietHoaDon
INNER JOIN HoaDon ON ChiTietHoaDon.SoHD = HoaDon.SoHD
INNER JOIN Khách ON HoaDon.MaK = Khách.MaK
GROUP BY Khách.MaK, Khách.TenK;

```

Giải thích:

- INNER JOIN: Kết nối các bảng ChiTietHoaDon, HoaDon, và Khách.
- SUM(ChiTietHoaDon.SoLuong * ChiTietHoaDon.DonGia): Tính tổng doanh thu của mỗi khách hàng.

- GROUP BY Khách.MaK, Khách.TenK: Nhóm dữ liệu theo mã khách hàng và tên khách hàng.

8.3.6. Kết luận

- GROUP BY giúp nhóm dữ liệu theo một hoặc nhiều cột và thực hiện các phép toán tổng hợp trên các nhóm dữ liệu.
- Kết hợp với các hàm tổng hợp như SUM(), COUNT(), AVG(), MAX(), MIN() để phân tích và báo cáo dữ liệu.
- Có thể kết hợp với HAVING để lọc kết quả nhóm và JOIN để kết hợp dữ liệu từ nhiều bảng.

Bài tập Thực hành:

- Tính tổng số lượng hàng hóa được bán ra của mỗi loại hàng hóa.
- Đếm số lượng hóa đơn của mỗi khách hàng.
- Tính doanh thu tổng của từng hóa đơn và sắp xếp theo doanh thu từ cao đến thấp.
- Tìm hàng hóa có doanh thu lớn nhất và nhỏ nhất trong mỗi hóa đơn.

8.4. Mệnh đề điều kiện đối với hàm nhóm - HAVING

8.4.1. Giới thiệu về HAVING

Trong SQL, Mệnh đề HAVING được sử dụng để lọc dữ liệu sau khi đã thực hiện nhóm dữ liệu với GROUP BY và áp dụng các hàm tổng hợp như SUM(), COUNT(), AVG(), MAX(), MIN().

Mệnh đề HAVING cho phép chúng ta đặt điều kiện cho các nhóm dữ liệu, giúp chúng ta chỉ giữ lại những nhóm thỏa mãn điều kiện cụ thể.

Một điểm khác biệt giữa HAVING và WHERE là trong điều kiện của WHERE không được có các hàm nhóm trong khi HAVING lại cho phép sử dụng các hàm nhóm trong điều kiện của mình.

8.4.2. Cú pháp của HAVING

SELECT [Danh sách các cột], [Hàm tổng hợp]

FROM [Tên bảng]

WHERE [Điều kiện lọc (nếu có)]

GROUP BY [Tên cột1], [Tên cột2], ...

HAVING [Điều kiện đối với hàm nhóm];

Trong đó:

- Danh sách các cột: Cột hoặc các cột chúng ta muốn hiển thị trong kết quả.
- Hàm tổng hợp: Các hàm như SUM(), COUNT(), AVG(), MAX(), MIN(), v.v.
- GROUP BY: Cột hoặc các cột dùng để nhóm dữ liệu.
- HAVING: Điều kiện lọc áp dụng cho các nhóm dữ liệu.

8.4.3. Ví dụ về HAVING

Sử dụng cơ sở dữ liệu quản lý bán hàng, gồm các bảng Hang, Khách, HoaDon, và ChiTietHoaDon.

Ví dụ 1: Đưa ra danh sách khách hàng có số lần mua hàng lớn hơn hoặc bằng 5.

```
SELECT Khách.MaK, Khách.TenK, COUNT(Khách.MaK) AS N'Số lần mua hàng'
```

```
FROM Khách, HoaDon
```

```
WHERE Khách.MaK = HoaDon.MaK
```

```
GROUP BY Khách.MaK, Khách.TenK
```

```
HAVING COUNT(Khách.MaK) >= 5
```

Ví dụ 2: Lấy danh sách các hàng hóa có tổng số lượng bán ra lớn hơn 100.

Câu lệnh SQL:

```
SELECT MaH, SUM(SoLuong) AS TotalQuantity
```

```
FROM ChiTietHoaDon
```

```
GROUP BY MaH
```

```
HAVING SUM(SoLuong) > 100;
```

Giải thích:

- SUM(SoLuong): Tính tổng số lượng bán ra của mỗi hàng hóa.
- GROUP BY MaH: Nhóm dữ liệu theo mã hàng hóa (MaH).
- HAVING SUM(SoLuong) > 100: Lọc các nhóm có tổng số lượng bán ra lớn hơn 100.

Ví dụ 3: Tính tổng doanh thu của mỗi hóa đơn và chỉ giữ lại những hóa đơn có tổng doanh thu lớn hơn 1000.

Câu lệnh SQL:

```
SELECT SoHD, SUM(SoLuong * DonGia) AS TotalRevenue
FROM ChiTietHoaDon
GROUP BY SoHD
HAVING SUM(SoLuong * DonGia) > 1000;
```

Giải thích:

- SUM(SoLuong * DonGia): Tính tổng doanh thu của mỗi hóa đơn.
- GROUP BY SoHD: Nhóm dữ liệu theo số hóa đơn (SoHD).
- HAVING SUM(SoLuong * DonGia) > 1000: Lọc các hóa đơn có tổng doanh thu lớn hơn 1000.

Ví dụ 4: Đếm số lượng hóa đơn của mỗi khách hàng và chỉ giữ lại những khách hàng có số lượng hóa đơn lớn hơn 5.

Câu lệnh SQL:

```
SELECT MaK, COUNT(SoHD) AS NumberOfInvoices
FROM HoaDon
GROUP BY MaK
HAVING COUNT(SoHD) > 5;
```

Giải thích:

- COUNT(SoHD): Đếm số lượng hóa đơn của mỗi khách hàng.
- GROUP BY MaK: Nhóm dữ liệu theo mã khách hàng (MaK).
- HAVING COUNT(SoHD) > 5: Lọc các khách hàng có số lượng hóa đơn lớn hơn 5.

Ví dụ 5: Tính giá trị trung bình của đơn giá cho mỗi hàng hóa và chỉ giữ lại những hàng hóa có giá trị trung bình đơn giá lớn hơn 50.

Câu lệnh SQL:

```
SELECT MaH, AVG(DonGia) AS AveragePrice
FROM ChiTietHoaDon
GROUP BY MaH
```

HAVING AVG(DonGia) > 50;

Giải thích:

- AVG(DonGia): Tính giá trị trung bình của đơn giá cho mỗi hàng hóa.
- GROUP BY MaH: Nhóm dữ liệu theo mã hàng hóa (MaH).
- HAVING AVG(DonGia) > 50: Lọc các hàng hóa có giá trị trung bình đơn giá lớn hơn 50.

Ví dụ 6. Kết hợp HAVING với JOIN

Yêu cầu: Lấy danh sách các khách hàng và tổng doanh thu của mỗi khách hàng, chỉ giữ lại những khách hàng có tổng doanh thu lớn hơn 5000.

Câu lệnh SQL:

```
SELECT  Khách.MaK,  Khách.TenK,  SUM(ChiTietHoaDon.SoLuong *  
ChiTietHoaDon.DonGia) AS TotalRevenue  
  
FROM ChiTietHoaDon  
  
INNER JOIN HoaDon ON ChiTietHoaDon.SoHD = HoaDon.SoHD  
  
INNER JOIN Khách ON HoaDon.MaK = Khách.MaK  
  
GROUP BY Khách.MaK, Khách.TenK  
  
HAVING SUM(ChiTietHoaDon.SoLuong * ChiTietHoaDon.DonGia) > 5000;
```

Giải thích:

- INNER JOIN: Kết nối các bảng ChiTietHoaDon, HoaDon, và Khách.
- SUM(ChiTietHoaDon.SoLuong * ChiTietHoaDon.DonGia): Tính tổng doanh thu của mỗi khách hàng.
- GROUP BY Khách.MaK, Khách.TenK: Nhóm dữ liệu theo mã khách hàng và tên khách hàng.
- HAVING SUM(ChiTietHoaDon.SoLuong * ChiTietHoaDon.DonGia) > 5000: Lọc các khách hàng có tổng doanh thu lớn hơn 5000.

Ví dụ 7. Kết hợp GROUP BY với HAVING

Yêu cầu: Lấy danh sách các hàng hóa có tổng số lượng bán ra lớn hơn 100.

Câu lệnh SQL:

```
SELECT MaH, SUM(SoLuong) AS TotalQuantity
```

FROM ChiTietHoaDon

GROUP BY MaH

HAVING SUM(SoLuong) > 100;

Giải thích:

- **HAVING SUM(SoLuong) > 100:** Lọc các nhóm có tổng số lượng bán ra lớn hơn 100.

8.4.4. Kết luận

- **HAVING** là một mệnh đề quan trọng để lọc dữ liệu sau khi nhóm và tính toán các hàm tổng hợp.
- Sử dụng **HAVING** giúp chúng ta tinh chỉnh kết quả phân tích và báo cáo dữ liệu theo yêu cầu cụ thể.
- **HAVING** chỉ có thể được sử dụng với các nhóm dữ liệu, không giống như **WHERE** vốn áp dụng cho các bản ghi đơn lẻ trước khi nhóm dữ liệu.

Câu hỏi

Câu 1. Trình bày cú pháp của lệnh truy vấn dữ liệu select? Cho ví dụ minh họa

Câu 2. Trình bày mệnh đề where và cách sử dụng trong câu lệnh truy vấn select? Cho ví dụ minh họa

Câu 3. Trình bày về mệnh đề group by trong câu lệnh truy vấn select? Cho ví dụ minh họa